

1.

不妨设 nut 和 bolt 分别从 1 到 n 编号, 标号越大型号越大.

(a) 考虑对手论证: 维护 $i = 1$ 和 $j = n$ 分别表示最小的未使用 nut 编号和最大的未使用 bolt 编号. 对于任意算法, 当它第一次询问到某个 nut, 为 nut 分配编号 i , 然后 $i \leftarrow i + 1$; 当它第一次询问到某个 bolt, 为 bolt 分配编号 j , 然后 $j \leftarrow j - 1$. 可见, 算法前 $n/2 = \Omega(n)$ 次询问得到的答案一定是 " $<$ ", 无法得到 nut-bolt 对. 因而最坏情况下, 找到一个 nut-bolt 对需要 $\Omega(n)$ 的时间.

(b) 考虑决策树深度估计, 不妨固定 nut 的顺序, bolt 共 $n!$ 种排列, 每种排列与 bolt 实际编号一一对应, 因此算法结束时对应的叶子结点数量不少于 $n!$. 算法每次询问至多将决策树分出三棵子树, 分别代表答案为 $< / = / >$ 时的子问题. 则树高不小于 $\Omega(\log_3 n!) = \Omega(n \log n)$.

2.

(a) 不妨设 $k = 2^t$. 算法每次 $\mathcal{O}(n)$ 地找到第 $n/2$ 大的元素, 然后递归两个 $(n/2, 2^{t-1})$ 规模的子问题. 复杂度 $\mathcal{O}(n \log k)$. 对于一般的 k , 只需要找第 $n/k \times \lfloor k/2 \rfloor + 1$ 大的元素即可, 论证同理.

(b) 和排序算法的决策树分析类似, 叶子至少有 $n!/((n/k)!)^k$ 个, 每次比较最多产生两个分支, 因而有复杂度下界 $\Omega(\log n! - k \log(n/k)!) = \Omega(n \log k)$.

(c) 对每个大小为 n/k 的子段调用排序算法即可, 复杂度 $\mathcal{O}(k \cdot n/k \cdot \log(n/k)) = \mathcal{O}(n \log(n/k))$.

(d) 同样分析决策树, 叶子至少有 $((n/k)!)^k$ 个, 每次比较最多产生两个分支, 因而有复杂度下界 $\Omega(n \log(n/k))$.

3.

(a) 考虑对手论证: 设当前算法进行过实验的最高楼层为 h , 初始时 $h = 0$. 当算法进行实验 h' , 只要 $h' > h + 1$, 回答 "碎", 这样算法无法确定地区分最低 unsafe 楼层 ($h + 1, \dots, h'$ 都有可能); 否则若 $h' = h + 1$, 回答 "没碎". 这样, 所有正确的确定性算法都必须依次从 1 询问至 L , 即至少询问 L 次.

对应的算法: 从低到高枚举楼层进行实验, 第一次碎掉时对应楼层就是 L . 复杂度 $\Theta(L)$, 这是渐进最优的算法.

(b) 算法设计: 第一枚蛋依次测试 1, 4, 9, $\dots$ 楼, 设它第一次在 k^2 楼碎掉, 则用第二枚蛋从 $(k-1)^2 + 1$ 楼依次测试到 k^2 楼, 碎掉时对应的楼层就是 L .

下界证明: 反证, 若存在一个询问次数 $f(L) = o(\sqrt{L})$ 的确定性算法, 当 $L \in [\ell, 2\ell]$ 足够大, 设算法的第一颗蛋在第 t_1 次询问时碎, 第二颗蛋在第 t_2 次询问时碎, 明显 (t_1, t_2) 决定了算法流程, 进而确定了算法输出. 但 (t_1, t_2) 只有 $\mathcal{O}(f(2\ell)^2) = o(\ell)$ 种, 不可能区分出 ℓ 种实际最低楼层, 矛盾.

因而确定性算法的询问次数下界为 $\Omega(\sqrt{L})$, 以上设计的算法是渐进最优的.